How the Portfolio Was Made

A closer look at the data-generating process behind the 12,000 businesses — what drives default, what only appears to, and the ceiling it puts on every model you will build.

For curious readers, after the session. Nothing here is needed to pass a checkpoint. Everything here is reproducible: the appendix has the code for every number on these pages. Source: `shared/data_generator.py` (195 lines — short enough to read in one sitting, and worth it).

> Session 1 compressed the whole generator into one arrow:
>
> `attributes → latent risk (logit) → + noise → Bernoulli draw → label`
>
> That arrow is correct, and it hides four things worth knowing: **noise enters twice, not once**; **eleven of your twenty usable columns have no causal role at all**; two of those eleven nonetheless carry real predictive signal; and the whole construction puts a hard, computable ceiling on model performance — one the committed scorecard has effectively already reached.

1 The shape of the thing

Two seeds, three tables, one dependency chain

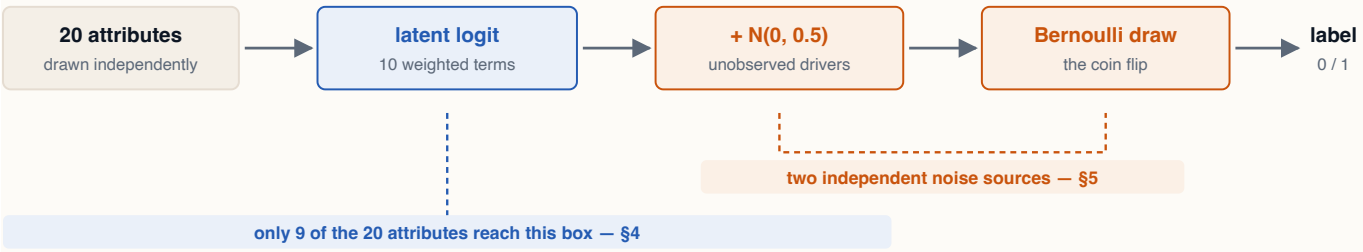
Everything is deterministic. `generate_businesses()` runs on `SEED = 42`; `generate_portfolio_and_panel()` runs on `seed + 1`. The split matters: the panel is generated *from* the booked subset, so the two cannot be re-run independently without desynchronising. `make data` always does both, which is why your numbers match the slides to the digit.

The three tables are not three datasets. They are one population, filtered twice:

THE DEPENDENCY CHAIN

Table	Rows	Is	Depends on
`businesses`	12,000	every applicant, with counterfactual outcomes	seed 42
`portfolio`	8,336	the subset where `booked = 1`	businesses + seed 43
`panel`	200,064	24 monthly rows per booked account	portfolio + seed 43

8,336 is not chosen. It falls out of an approval model applied to 12,000 applicants, which is how a real book accumulates — and it is why the book defaults at 10.9% while the people who applied default at 16.7%.



The generator, with the two things the one-line version omits. Noise is injected at two separate points, and the arrow into the logit is far narrower than it looks — less than half the attribute table gets through it.

2 Stage one — the marginals

Each attribute drawn from a named distribution, chosen for shape

The first 25 lines of the generator draw attributes one at a time, independently. Each distribution was picked so the *shape* is recognisable to anyone who has looked at a real SME book:

SELECTED MARGINALS — DATA_GENERATOR.PY LINES 38–56

Attribute	Draw	Why that shape
`years_in_business`	`gamma(3, 3)`	right-skewed; most SMEs are young, a few are 40 years old
`annual_revenue`	`lognormal(13, 1)`	the long tail — median $437k, mean $736k, max $24.8M
`employees`	`lognormal(2, 1)`	median 7 people; this is a small-business book
`dscr`	`normal(1.4, 0.5)`	centred just above the 1.0 covenant line, so both sides are populated
`utilization`	`beta(2, 3)`	naturally bounded 0–1, with mass toward the low end
`prior_delinquencies`	`poisson(0.6)`	count data, mostly zero — 54% of the book has none
`public_records`	`binomial(1, 0.08)`	an 8% base rate for liens, judgments, filings

Three attributes are derived, not drawn. This is the only place correlation enters the applicant table:

```
# the entire correlation structure of the dataset, in three lines
net_income       = annual_revenue * profit_margin
leverage         = total_debt / annual_revenue
requested_amount = annual_revenue * uniform(0.05, 0.50)
```

Remember those three lines. Section 4 is almost entirely about their consequences.

3 Stage two — the latent logit

Ten terms, and why the largest weight is nearly the smallest effect

Risk is assembled as a linear score on the log-odds scale. Most terms are standardised through `_z()` first, so their weights are broadly comparable:

```
logit = -2.2
      - 0.60·z(log years)      - 0.50·z(log revenue)   - 0.70·z(dscr)
      - 0.40·z(current_ratio)  + 0.60·z(leverage)     + 0.55·z(utilization)
      + 0.50·z(prior_delinq)   + 0.70·public_records - 0.40·z(log credit_history)
      - 1.50·profit_margin
      + normal(0, 0.5)          # <- noise source one
```

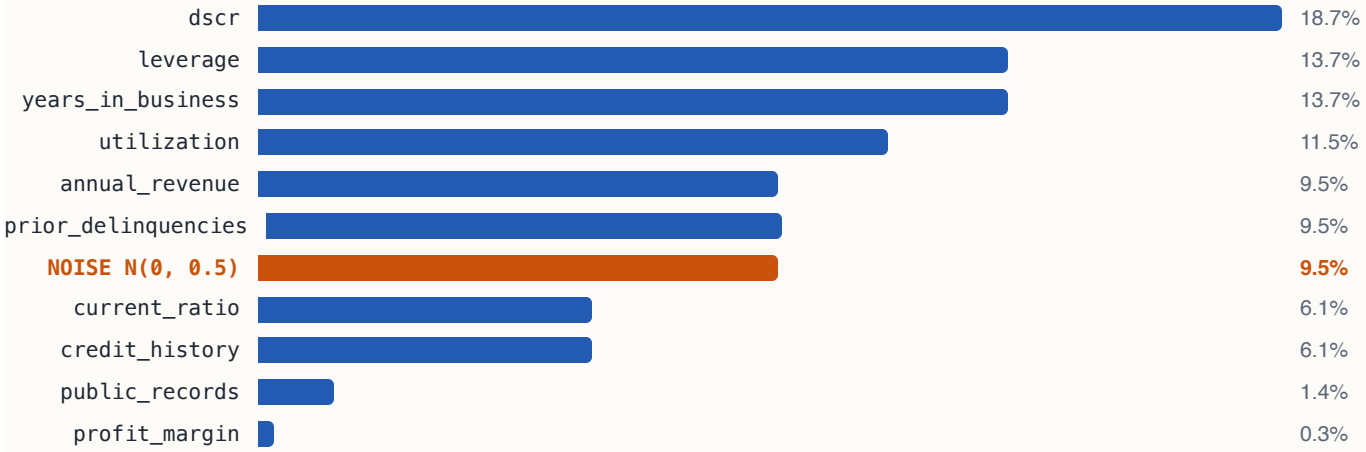
The intercept of −2.2 sets the base rate. Signs encode credit intuition: more coverage and more history push risk down, more leverage and more utilization push it up.

A WEIGHT IS NOT AN EFFECT SIZE

`profit_margin` carries **−1.50**, the largest coefficient in the equation, and contributes **0.3%** of the variance in risk. It is one of two terms that is *not* standardised, and its raw spread is about 0.06. Multiply through and its effective swing is ~0.09 log-odds — against 0.70 for `dscr`.

A coefficient becomes an effect only when you multiply it by how much the variable actually moves. This is worth carrying into every model review you ever do.

Decomposing the logit into the share of variance each term contributes gives the honest ranking:



Share of latent-risk variance, by term. The orange bar is not a feature — it is the injected noise, and it outranks four of the ten real drivers. Signal std 1.545 against noise std 0.499 — drawn as 0.5, recovered from the data at 0.499.

Two readings of that chart matter. First, `dscr` is the single most important driver — which is why Session 3's adjudication policy puts a hard floor on it, and why your Session 2 scorecard will lean on it.

Second, **the noise term is the seventh-largest contributor to risk.** It is not a rounding detail. It is a design decision with consequences that run to Section 6.

4 What is *not* in the logit

Eleven columns with no causal role — and why two of them predict anyway

Your feature scan produced twenty candidates. The logit touches nine of them. The other eleven have **no causal effect on default whatsoever**. But they fail in two completely different ways, and telling them apart is the transferable skill in this whole note.

Case one: structural zeros

Eight columns are absent from the logit *and* uncorrelated with anything in it: `industry`, `region`, `entity_type`, `loan_purpose`, `term_months`, `collateral_flag`, `trade_lines`, `employees`. Their true effect is exactly zero, and their measured Information Value — 0.0005 to 0.006 — is the residue of finite sampling.

The Session 1 notebook makes this concrete with a chart you have already seen: default rate by industry, ten bars, a clean 3.3-point spread from Wholesale at 15.1% to Construction at 18.4%, in an order that invites a story. Construction is cyclical. Hospitality is fragile. It reads as a finding.

> **THE UNCOMFORTABLE PART**
>
> The data was not misleading. The chart was an accurate rendering of 12,000 rows. The *story* — the mechanism that made the spread feel real — was supplied by the reader, and it fit the noise perfectly.
>
> You cannot fix this with better data hygiene, because nothing was wrong with the data. You fix it by requiring a number before you accept a story: an IV of 0.006 against a "usable" floor of 0.02 settles in one line what an afternoon of segment discussion will not.

Case two: inherited proxies — the dangerous one

Two columns are absent from the logit but carry **real, medium-strength, monotonic IV**. They inherit it from `annual_revenue`, which is a genuine driver, through the three derived-column lines in Section 2.

SIGNAL WITH NO CAUSE

Column	IV	Rank-corr with revenue	Where the signal comes from
`requested_amount`	0.103	+0.85	`= revenue × uniform(0.05, 0.50)`
`net_income`	0.100	+0.65	`= revenue × profit_margin`
`total_debt`	0.014	+0.81	enters risk only as the ratio `leverage`

`requested_amount` is the one to remember. On a feature scan it looks like a keeper: medium strength, cleanly monotonic, comfortably above every screening threshold. It will survive your train/test split. It will hold up on a holdout. It will still be there next quarter.

And it has **no causal relationship to default at all.** It works for exactly as long as the relationship between what a business earns and what it asks for stays where it is.

WHY A BORROWED FEATURE IS WORSE THAN A WEAK ONE

A weak feature is harmless: you drop it and lose nothing. A borrowed feature fails *silently and late*.

Change the product — a campaign pushing larger asks, a new minimum ticket, a segment that borrows differently against the same revenue — and the borrowed signal drains out of the feature while every drift monitor you own stays green. The distribution of `requested_amount` has not moved. Its *relationship to the thing that actually drives risk* has, and nothing in a standard monitoring pack is watching that.

Session 2's scorecard drops both proxies and keeps `annual_revenue`: take the driver, not the shadow. That is defensible, and it is worth being clear that it was **a judgement, not an output**. Nothing in the feature scan told anyone to do it — on IV alone, `requested_amount` outranks `credit_history_months`, which the scorecard keeps.

On real data you cannot open the generator and check. You have the correlation, the domain argument, and your judgement. The question is the same either way, and it is the one to take back to work:

THE QUESTION

If this feature works, what is the mechanism — and what would have to change for it to stop working?

A feature that cannot answer the first half is a proxy you have not identified yet. A feature that cannot answer the second half has never been stress-tested. Here, uniquely, you can practise the question with the answer key open on the desk. That is the entire reason this data is synthetic.

5 Stage three — noise enters twice

Two independent sources, doing two different jobs

"+ noise" on the slide covers two mechanisms. They are not interchangeable.

(a) The latent noise — `normal(0, 0.5)`

Added inside the logit, before any probability exists. It contributes 9.5% of risk variance (recovered from the data at std 0.499 against a signal std of 1.545).

Conceptually this is **real risk drivers you did not observe**: the owner's health, a customer concentration nobody disclosed, a lawsuit filed next March. It genuinely moved the outcome. You simply had no column for it. No feature engineering recovers it, because the information was never in the table.

(b) The Bernoulli draw — `rng.binomial(1, pd_true)`

This is the larger effect, and the one people find hardest to accept.

The label is not a threshold on PD. It is a **coin flip weighted by** PD. A borrower at PD 0.30 defaults 30% of the time — meaning 70% of your worst-scored applicants repay perfectly, and always will, no matter how good your model is.

```
mean pd_default_origination = 0.1696
realised default rate       = 0.1672   # the draw, not the formula
```

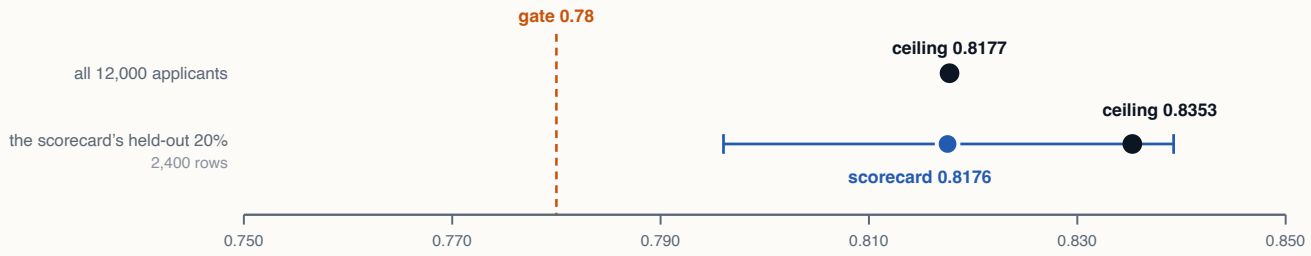
This is the design rule Session 1 states as "*the label is a draw, not a formula.*" It is also the reason perfect prediction is impossible **by construction** rather than by accident — and it sets up the only number in this note that should change how you work.

6 The ceiling this creates

Where the gate came from, and why 0.82 is not "room for improvement"

Because the generator is known, the maximum achievable performance can be computed rather than guessed. Score the noise-free signal — the logit with its noise term removed — against the labels that were actually drawn. Over all 12,000 applicants that gives **AUC 0.8177**: the best any model using observable features could possibly do. An omniscient observer who could also see the injected noise would reach 0.8311, and no one can.

Comparing that with a trained model takes care, because both numbers must be measured on the *same rows*. The committed scorecard reports 0.8176 on its held-out 20%. Recomputing the ceiling on **that same split** gives **0.8353** — those particular 2,400 rows happen to be easier to rank than the population as a whole.



The ceiling depends on which rows you measure it over. The blue bar is the 95% sampling interval on an AUC estimated from 2,400 rows (±0.02, bootstrapped). It covers the ceiling measured on those same rows — so on the evidence available, the scorecard and the theoretical maximum **cannot be told apart**. The gate sits about 0.04 below the population ceiling.

THE NUMBERS, WITH THEIR SAMPLES ATTACHED

Quantity	Measured over	AUC
Ceiling — noise-free signal	all 12,000 applicants	0.8177
Ceiling — true PD, incl. the noise	all 12,000 applicants	0.8311
Ceiling — noise-free signal	the scorecard's held-out 20%	0.8353
Committed scorecard	the scorecard's held-out 20%	0.8176
The gate asserted in `train.py`	the same held-out 20%	0.7800

On the same rows, then, the scorecard sits **0.018 below its ceiling** — and the 95% interval on a 2,400-row AUC is about ±0.02. The gap is inside the noise of the measurement. The honest statement is not "0.018 short of the maximum"; it is "**indistinguishable from the maximum, and this much test data cannot resolve the difference.**"

A 20% SPLIT BUYS YOU ±0.02 — AND THAT IS ITS OWN LESSON

The ceiling moved from 0.8177 to 0.8353 purely by changing which 2,400 rows it was computed over. Nothing about the model, the features or the data changed. Only the sample did.

Carry that number back to work. Two models whose held-out AUCs differ by 0.01 on a test set this size have not been distinguished, and the third decimal place in a model report is decoration. If a decision rests on a difference that small, it needs cross-validation or a bootstrap interval — not a bigger table.

Two caveats on the ceiling itself, stated precisely. It involves **no fitting** — it scores a known quantity against known labels — so overfitting does not apply to it and computing it over the full population is legitimate. And it is a property of *this* data: change the generator's noise term and every number in this section moves.

WHAT THIS CHANGES

The scorecard is not "a good model with room to improve." It is **at, or indistinguishably near, the wall**. The 0.18 of AUC that remains is unreachable by anyone, with any method, ever — it is the Bernoulli draw, and no amount of gradient boosting touches it.

Which reframes what a metric target *is*. A gate is not an ambition. It is a claim about what this data can support, and it should be set after measuring the ceiling — never before. Set a gate above the ceiling and you have not been demanding; you have ordered something impossible, and someone will eventually deliver it by leaking.

On synthetic data the ceiling is computable. On real data it still exists — you just cannot see it, which is worse. Sessions 4 and 5 return to this.

7 The three downstream labels

Where the book, the watchlist and the line-increase targets come from

booked — the approval filter

```
approve_logit = 2.0 - 6.0 * pd_true + normal(0, 0.5)
booked        = binomial(1, sigmoid(approve_logit))
```

A steep but deliberately **soft** function of true PD. The bank is good at declining bad risk, not perfect at it, and the noise term is what leaves genuinely bad accounts on the book — without which no early-warning problem would exist in Session 4.

This one line produces the entire selection-bias story: 10.9% default on the book against 29.9% among those who never reached it, and a mean DSCR of 1.46 versus 1.28. It is also why the dataset can show you something a real book never will — the counterfactual outcomes of the applicants you turned away.

`deterioration_next_6_12mo` — the Session 4 target

Driven by `pd_true` plus two quantities computed *from the panel*: mean utilization over the last three months above 0.85, and the utilization trend across all 24 months. The behavioural panel must therefore exist before this label can — which is why Session 4's features are trajectory-shaped rather than snapshots, and why its gate is written on top-decile capture rather than AUC.

`line_increase_good` — and an honest note

Its noise term is deliberately smaller (`normal(0, 0.18)`, against 0.4 and 0.5 elsewhere) and it leans harder on observable on-book behaviour. The comment above it in the source says why:

```
# The reference build's first version of this label was noise-capped below its
# metric gate — the fix (and when such a fix is safe at all) is a Session 4 story.
```

Someone raised a label's signal-to-noise ratio so that a model could clear a bar. Whether that is legitimate depends entirely on whether the bar or the label was the thing that was wrong — and it is left in the open, in a comment, rather than quietly smoothed over. That is the behaviour worth copying.

8 What this generator deliberately does not do

The limitations, stated first

A room of risk practitioners will find these. Better to name them:

- **The risk drivers are mutually independent.** Every pairwise correlation among `dscr`, `leverage`, `current_ratio`, `utilization`, `years_in_business`, `credit_history_months` and `prior_delinquencies` is under $|0.025|$. In reality DSCR and leverage are mechanically linked. **This is the single largest departure from a real book**, and it makes the scorecard's job unrealistically clean — no multicollinearity, no unstable coefficients, no argument about which of two correlated features to keep.
- **No macro dimension and no time at origination.** No vintages, no cycle, no downturn. Every applicant is drawn from one stationary world. This is part of why segment features carry nothing: there is no shock for an industry to be differentially exposed to.
- **No missingness.** Which is why the notebook's null audit is a pure bug check rather than an imputation exercise — and why nothing in the workshop teaches imputation.
- **The panel is a random walk with drift, not a behavioural model.** Utilization drifts by `(pd_true - mean) × 0.015` per month plus `normal(0, 0.03)`; balances, days past due, deposits and overdrafts are then deterministic functions of that path. Real accounts have seasonality, paydown cycles and step changes. This one does not.

Each of these makes the platform easier than reality. None of them makes it dishonest, because they are written down. A synthetic dataset whose limitations are documented is a teaching instrument; the same dataset with the limitations unstated is a misleading one.

9 What to take back to work

1. **Multiply the coefficient by the spread.** The largest weight in this model contributes 0.3% of its variance. Weight tables mislead unless standardised, and most are not.
2. **Get a number before you accept a story.** The industry chart was accurate, the story was invented, and one IV calculation settled it.

3. **Ask what mechanism makes a feature work.** A feature that predicts without a mechanism is borrowing signal from something you have not identified, and it will fail silently when the loan relationship changes rather than when the feature does.
4. **Monitor relationships, not just distributions.** Every drift check on `requested_amount` would stay green through the exact failure that breaks it.
5. **Measure the ceiling before you set the target.** On synthetic data you can compute it. On real data you cannot — which is an argument for humility about targets, not for ignoring them.

A Appendix — reproduce every number here

Nothing in this note should be taken on trust

Run from the repo root at `stage-1` or later. The first block reconstructs the signal-only logit and computes the two ceilings; the second reproduces the variance decomposition in Section 3.

```
import numpy as np, pandas as pd
from sklearn.metrics import roc_auc_score
from shared.config import RAW

b = pd.read_parquet(RAW / "businesses.parquet"); y = b["default"].to_numpy()
z = lambda x: (np.asarray(x, float) - np.mean(x)) / np.std(x)
pm = np.clip(b["net_income"] / np.maximum(b["annual_revenue"], 1), -1, 1)

# the DGP's logit, with the noise term left out
signal = (-2.2
    - 0.60*z(np.log(b["years_in_business"])) - 0.50*z(np.log(b["annual_revenue"]))
    - 0.70*z(b["dscr"])                      - 0.40*z(b["current_ratio"])
    + 0.60*z(b["leverage"])                  + 0.55*z(b["utilization"])
    + 0.50*z(b["prior_delinquencies"].astype(float)) + 0.70*b["public_records"]
    - 0.40*z(np.log(b["credit_history_months"])) - 1.50*pm).to_numpy()

print(roc_auc_score(y, signal))                          # 0.8177  population ceiling
print(roc_auc_score(y, b["pd_default_origination"]))     # 0.8311  omniscient

# the ceiling on the SAME rows the scorecard was scored on — the only fair comparison
from sklearn.model_selection import train_test_split
from shared.config import SEED
_, te = train_test_split(np.arange(len(b)), test_size=0.2, random_state=SEED, stratify=y)
print(roc_auc_score(y[te], signal[te]))                  # 0.8353  same-split ceiling

# recover the injected noise: full logit minus the signal you just rebuilt
p = b["pd_default_origination"].to_numpy()
noise = np.log(p / (1 - p)) - signal
print(signal.std(), noise.std())                         # 1.545  0.499
```

The proxy claims in Section 4, and the scorecard's committed metrics:

```
# rank-correlation, because IV is computed on bins
for c in ["requested_amount", "net_income", "total_debt"]:
    print(c, b[c].corr(b["annual_revenue"], method="spearman").round(2))

# the driver-independence claim in section 8
print(b[["dscr","leverage","current_ratio","utilization"]].corr().round(3))

# the committed scorecard's numbers (stage-2 or later)
import json; d = json.load(open("score/models/metadata.json"))
print(d["metrics"]["auc"], d["gate"])              # 0.8176  {'auc_min': 0.78}
```

The Session 1 notebook (`notebooks/01_stage1_portfolio.ipynb`) computes the feature scan, the industry case and the proxy case interactively. The oracle-ceiling calculator for Session 4's target lives at `workshop/labs/oracle_ceiling.py` and does for `deterioration` what the appendix block above does for `default`.

If a number here disagrees with what your machine prints, your machine is right and this note is a bug — please say so.